Améliorer la qualité du code - les tests unitaires

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Contexte **3**

II. Le test unitaire **3**

A. Le test unitaire 3

B. Exercice : Quiz 5

III. Le Test-Driven Development (TDD) **7**

A. Le Test-Driven Development (TDD) 7

B. Exercice : Quiz 10

IV. Introduction à Gherkin et aux tests d'acceptation **12**

A. Introduction à Gherkin et aux tests d'acceptation 12

B. Exercice : Quiz 14

V. Essentiel **16**

VI. Auto-évaluation **16**

A. Exercice 16

B. Test 16

Solutions des exercices **17**

Tous droits réservés à STUDI - Reproduction interdite

I. Contexte

Contexte

Imaginons que nous préparons un gâteau. Avant de commencer à mélanger tous les ingrédients, nous nous assurons que chaque élément nécessaire est présent et de bonne qualité. Nous vérifions que la farine est fraîche, que la levure n'est pas périmée, que les œufs sont frais, et que le four est bien préchauffé. Ces petites vérifications isolées, simples et rapides, sont souvent négligées mais permettent pourtant d'éviter à recommencer intégralement l'opération ou jeter le tablier.

Les tests unitaires dans le développement logiciel fonctionnent de manière similaire. Avant de lancer un programme complet, on vérifie que chaque « *ingrédient* » du code, qu'il s'agisse d'une fonction ou d'une méthode, fait exactement ce qu'il doit faire de manière indépendante. Cela permet de détecter rapidement des problèmes avant qu'ils n'affectent le reste du programme.

Tout comme il serait désastreux de découvrir au dernier moment que la farine est périmée après avoir mélangé tous les ingrédients, il serait coûteux en temps et en effort de découvrir qu'une petite partie du code cause des erreurs après l'assemblage de l'ensemble du programme. En effectuant des tests réguliers sur chaque composant du code, les tests unitaires, permettent de garantir que chaque pièce fonctionne comme prévu, contribuant ainsi à la solidité et la fiabilité de l'ensemble.

II. Le test unitaire

A. Le test unitaire

Définition **Qu'est-ce qu'un test unitaire ?**

Un **test unitaire** est un test qui vérifie le bon fonctionnement d'une petite unité de code, telle qu'une fonction ou une méthode, de manière **isolée**.

Imaginons que nous construisons un meuble en kit. Avant d'assembler toutes les pièces ensemble, on vérifie individuellement chaque vis, chaque planche pour s'assurer qu'elles sont en bon état. Un test unitaire agit de manière similaire : il s'assure que chaque pièce du programme fonctionne correctement avant de l'assembler au reste du système. Cela garantit que, même pris en isolément, chaque composant fait ce qu'il est censé faire.

Un test unitaire est souvent automatisé pour pouvoir être exécuté régulièrement et rapidement. Il aide à identifier les erreurs dès que des modifications sont apportées au code. L'unité testée doit être isolée, ce qui signifie que l'environnement du test doit être contrôlé et que les dépendances externes (comme des bases de données, des services web, ou des fichiers) doivent être simulées ou ignorées. Cela garantit que les résultats des tests reflètent fidèlement le comportement de l'unité seule, sans être affectés par des facteurs externes.

Exemple

Prenons l'exemple d'une fonction simple qui additionne deux nombres. Pour tester cette fonction, il est nécessaire d'écrire un test unitaire qui vérifiera si cette addition retourne les résultats attendus pour différents ensembles de nombres. Cela permet d'assurer que la fonction se comporte correctement dans tous les cas testés.

```python
# Exemple de fonction à tester
def addition(a, b):
    return a + b
```

Dans ce cas, le test unitaire vérifiera si, en fournissant des entrées comme `2` et `3`, la fonction retourne le résultat attendu, qui est `5`. Si ce test est validé, nous savons que l'addition fonctionne pour ces valeurs. Si nous modifions la fonction dans le futur, ce test nous permettra de détecter rapidement tout problème introduit.

Testons ensemble la fonction `addition(a, b)` définie précédemment. Pour cela, nous allons écrire un test unitaire qui validera que cette fonction retourne bien la somme des deux nombres fournis en entrée.

```python
import unittest

# La fonction que nous voulons tester
def addition(a, b):
    return a + b

# Classe de test
class TestAddition(unittest.TestCase):

    def test_addition_positive_numbers(self):
        result = addition(2, 3)
        self.assertEqual(result, 5)

    def test_addition_negative_numbers(self):
        result = addition(-2, -3)
        self.assertEqual(result, -5)

    def test_addition_mixed_numbers(self):
        result = addition(-2, 3)
        self.assertEqual(result, 1)

# Exécution des tests
if __name__ == "__main__":
    unittest.main()
```

Dans cet exemple :

1. Le premier test, `test_addition_positive_numbers`, vérifie que l'addition de deux nombres positifs (`2 + 3`) donne bien `5`.
2. Le second test, `test_addition_negative_numbers`, s'assure que l'addition de deux nombres négatifs (`-2 + -3`) donne `-5`.
3. Le troisième test, `test_addition_mixed_numbers`, vérifie que l'addition d'un nombre négatif et d'un nombre positif retourne la bonne somme (`-2 + 3 = 1`).

Prenons un autre exemple : une fonction qui vérifie si un nombre est pair.

```python
# Fonction à tester
def est_pair(n):
    return n % 2 == 0

# Classe de test
class TestEstPair(unittest.TestCase):

    def test_nombre_pair(self):
        self.assertTrue(est_pair(4))  # 4 est un nombre pair

    def test_nombre_impair(self):
        self.assertFalse(est_pair(3))  # 3 est un nombre impair

# Exécution des tests
if __name__ == "__main__":
    unittest.main()
```

Ici, nous avons deux tests :

- `test_nombre_pair` vérifie que la fonction `est_pair` retourne `True` pour le nombre pair `4`.
- `test_nombre_impair` s'assure que la fonction retourne `False` pour le nombre impair `3`.

Tous droits réservés à STUDI - Reproduction interdite

Ces tests montrent comment une fonction qui effectue une opération simple peut être testée pour garantir son bon fonctionnement dans différentes conditions.

L'importance des tests unitaires

Les tests unitaires offrent plusieurs avantages importants dans le processus de développement logiciel. Ils permettent non seulement de garantir la qualité et la stabilité du code, mais aussi d'améliorer l'efficacité du développement à long terme :

- **Ils identifient rapidement les bugs dans une fonction ou une méthode spécifique**. En exécutant des tests après chaque modification du code, les développeurs peuvent détecter immédiatement toute régression ou erreur introduite par les changements. Cette approche proactive permet de réduire les risques de bugs en production.
- Lorsque le code doit être refactorisé (c'est-à-dire réorganisé pour améliorer sa lisibilité, sa structure ou ses performances), les tests unitaires **garantissent que les fonctionnalités existantes continuent de fonctionner correctement**. Le développeur peut ainsi modifier le code en toute confiance, sachant que les tests détecteront toute anomalie.
- Une régression se produit lorsqu'un bug déjà résolu réapparaît à la suite d'une modification du code. Grâce aux tests unitaires, il est possible de **vérifier que les fonctionnalités qui fonctionnaient auparavant continuent de fonctionner après chaque mise à jour**. Cela assure une meilleure stabilité du logiciel au fur et à mesure qu'il évolue.

B. Exercice : Quiz

[solution n°1 p.19]

Question 1

Quel est l'objectif principal d'un test unitaire ?

- ○ Tester l'intégralité du système
- ○ Vérifier qu'une unité isolée de code fonctionne correctement
- ○ Vérifier la performance du programme
- ○ Tester l'interaction entre plusieurs modules

Question 2

Quelle bibliothèque est principalement utilisée pour écrire des tests unitaires en Python ?

- ○ NumPy
- ○ unittest
- ○ Flask
- ○ Pytest

Question 3

Pourquoi les tests unitaires sont-ils importants ?

- ○ Ils permettent d'optimiser le code
- ○ Ils facilitent la détection des bugs à un stade précoce
- ○ Ils augmentent les performances du programme
- ○ Ils réduisent le temps de développement

Tous droits réservés à STUDI - Reproduction interdite

Question 4

Qu'est-ce que l'instruction assertEqual dans unittest vérifie ?

- Que deux valeurs sont différentes
- Que deux valeurs sont égales
- Que la fonction soulève une exception
- Que la fonction retourne toujours True

Question 5

Quel est le meilleur moment pour exécuter les tests unitaires ?

- Seulement avant de livrer le code en production
- À chaque modification du code
- Une fois que toutes les fonctionnalités sont implémentées
- Avant de commencer à écrire du code

Question 6

Que se passe-t-il si un test unitaire échoue ?

- Le programme continue à s'exécuter normalement
- Il lève une exception pour signaler qu'une partie du code ne fonctionne pas comme prévu
- Le test est ignoré
- Le code entier est supprimé

Question 7

Quelle est la bonne pratique pour un test unitaire efficace ?

- Tester plusieurs fonctionnalités à la fois
- Tester chaque fonctionnalité individuellement
- Ignorer les dépendances externes
- Utiliser le même test pour plusieurs fonctions

Question 8

Pourquoi est-il important de maintenir les tests unitaires à jour ?

- Parce que les tests doivent refléter les changements dans le code
- Parce que les tests obsolètes sont automatiquement supprimés
- Pour augmenter la vitesse des tests
- Parce que Python exige des tests unitarisés pour chaque nouvelle ligne de code

Question 9

Tous droits réservés à STUDI - Reproduction interdite

Que signifie la notion de « *couverture de code* » dans les tests unitaires ?

- ○ Le pourcentage de lignes de code qui ont été exécutées pendant les tests
- ○ Le nombre total de tests unitaires dans un projet
- ○ La durée totale d'exécution des tests
- ○ Le nombre d'exceptions gérées par le programme

Question 10

Quelle est la bonne façon de structurer un test unitaire ?

- ○ Appeler directement toutes les fonctions dans le test principal
- ○ Préparer les données, exécuter la fonction à tester, puis valider le résultat
- ○ Tester plusieurs fonctions dans le même test pour gagner du temps
- ○ Ignorer les valeurs d'entrée et se concentrer uniquement sur le résultat

III. Le Test-Driven Development (TDD)

A. Le Test-Driven Development (TDD)

Définition

Le Test-Driven Development (TDD) est une méthode de développement logiciel où l'écriture de tests unitaires précède l'implémentation du code. En suivant cette approche, le développement est guidé par les tests : on commence par définir les attentes sous forme de tests, puis on écrit juste assez de code pour que ces tests passent, et enfin, on améliore le code (refactorisation) en s'assurant que les tests restent valides.

Déroulement du TDD

Le cycle du TDD repose sur trois étapes fondamentales :

- **Red (Rouge)** : on écrit un test pour une fonctionnalité qui n'est pas encore implémentée. Ce test échoue puisque le code pour le faire fonctionner n'existe pas encore.
- **Green (Vert)** : on écrit le code minimal nécessaire pour que le test passe. L'objectif est simplement de réussir le test, sans se soucier de la qualité du code.
- **Refactor (Refactorisation)** : une fois que le test passe, on améliore le code tout en s'assurant que le test continue de fonctionner. On peut nettoyer le code, améliorer sa lisibilité ou ses performances.

Contrairement au développement traditionnel, où l'on écrit du code puis on teste ensuite, le TDD oblige à écrire d'abord le test. C'est comme préparer un plan avant de construire une maison. Vous définissez d'abord ce que vous voulez que le programme fasse, puis vous écrivez juste assez de code pour atteindre cet objectif.

Exemple **Implémentation du TDD**

Voici un exemple plus complexe de code utilisant Test-Driven Development (TDD), José a besoin de nous pour implémenter un système de gestion de comptes bancaires, avec des opérations comme le dépôt, le retrait, et la vérification du solde.

Nous allons implémenter une classe `CompteBancaire` qui permet à un utilisateur d'effectuer des dépôts, des retraits, et de vérifier le solde de son compte. La méthode TDD sera suivie : écriture des tests en premier, puis ajout du code pour faire passer les tests, et enfin la refactorisation.

Tous droits réservés à STUDI - Reproduction interdite

Étape 1 : Écriture des tests (Phase Red)

Nous commençons par écrire les tests unitaires pour les fonctionnalités suivantes :

- **Création d'un compte bancaire avec un solde initial**
- **Dépôt d'une somme sur le compte**
- **Retrait d'une somme du compte**
- **Vérification du solde**

Ces fonctionnalités ne sont pas encore implémentées, donc les tests échoueront dans un premier temps.

```python
import unittest

class TestCompteBancaire(unittest.TestCase):

    def test_creation_compte(self):
        compte = CompteBancaire("José", 100)
        self.assertEqual(compte.solde, 100)

    def test_depot(self):
        compte = CompteBancaire("José", 100)
        compte.depot(50)
        self.assertEqual(compte.solde, 150)

    def test_retrait(self):
        compte = CompteBancaire("José", 100)
        compte.retrait(30)
        self.assertEqual(compte.solde, 70)

    def test_solde_insuffisant(self):
        compte = CompteBancaire("José", 100)
        with self.assertRaises(ValueError):
            compte.retrait(200)

    def test_verification_solde(self):
        compte = CompteBancaire("José", 100)
        self.assertEqual(compte.get_solde(), 100)

# Exécution des tests
if __name__ == "__main__":
    unittest.main()
```

Étape 2 : Implémentation minimale pour faire passer les tests (Phase Green)

Maintenant que les tests sont définis, nous devons implémenter la classe `CompteBancaire` et ses méthodes pour que les tests passent. Voici une première implémentation simple qui se concentre sur le fait de faire passer les tests.

```python
class CompteBancaire:
    def __init__(self, titulaire, solde_initial):
        self.titulaire = titulaire
        self.solde = solde_initial

    def depot(self, montant):
        self.solde += montant

    def retrait(self, montant):
        if montant > self.solde:
            raise ValueError("Solde insuffisant")
```


Tous droits réservés à STUDI - Reproduction interdite

```
        self.solde -= montant

    def get_solde(self):
        return self.solde
```

Faisons un point sur les tests de l'étape 1 :

- Le test `test_creation_compte` passe car la classe `CompteBancaire` est initialisée avec un solde.
- Le test `test_depot` passe car la méthode `depot` additionne correctement la somme au solde.
- Le test `test_retrait` passe car la méthode `retrait` soustrait le montant du solde.
- Le test `test_solde_insuffisant` passe car la méthode `retrait` lève une exception lorsque le retrait dépasse le solde.
- Le test `test_verification_solde` passe car la méthode `get_solde` retourne correctement le solde actuel.

Étape 3 : Refactorisation (Phase Refactor)

Maintenant que tous les tests passent, nous pouvons refactoriser le code pour le rendre plus lisible, ou pour améliorer son organisation.

Nous allons améliorer l'organisation du code, ajouter des validations supplémentaires dans les méthodes pour s'assurer que les dépôts et les retraits sont toujours des montants valides (positifs), et rendre les erreurs plus explicites.

```
class CompteBancaire:
    def __init__(self, titulaire, solde_initial=0):
        self.titulaire = titulaire
        if solde_initial < 0:
            raise ValueError("Le solde initial ne peut pas être négatif")
        self.solde = solde_initial

    def depot(self, montant):
        if montant <= 0:
            raise ValueError("Le montant du dépôt doit être positif")
        self.solde += montant

    def retrait(self, montant):
        if montant <= 0:
            raise ValueError("Le montant du retrait doit être positif")
        if montant > self.solde:
            raise ValueError("Solde insuffisant pour effectuer ce retrait")
        self.solde -= montant

    def get_solde(self):
        return self.solde
```

Dans l'étape 3, nous avons créé 3 éléments :

- **Validation du solde initial** : dans le constructeur, on vérifie que le solde initial ne peut pas être négatif.
- **Validation des montants de dépôt et de retrait** : les montants doivent être positifs. Si l'utilisateur tente de déposer ou de retirer un montant inférieur ou égal à 0, une exception `ValueError` est levée.
- **Amélioration des messages d'erreur** : les erreurs sont plus explicites, ce qui facilite la détection des problèmes lors de l'utilisation de la classe.

Exemple **Cas pratique**

Tous droits réservés à STUDI - Reproduction interdite

Remarque

La tentation naturelle serait de coder d'abord et de tester après. Cependant, en TDD, il faut respecter l'ordre : on écrit d'abord les tests, même si cela peut sembler contre-intuitif au début. Cela force les développeurs à réfléchir dès le départ à la manière dont leur code sera testé, ce qui améliore la conception logicielle.

Fondamental **Avantage du TDD**

Le TDD nous oblige à penser dès le début à la manière dont le code sera testé. Cela conduit naturellement à une meilleure conception du code. Un code conçu pour être testé est souvent plus simple, modulaire et plus facile à maintenir. Comme chaque fonctionnalité doit être validée par un test, le développeur est poussé à écrire des fonctions plus petites, plus indépendantes et plus cohérentes.

Imaginons que nous développons une fonction de calcul dans une classe `Calculatrice`. En suivant le TDD, nous allons d'abord écrire un test pour la fonction `addition`. Cela poussera à écrire une fonction simple, facile à tester et indépendante des autres fonctionnalités.

Le TDD encourage une qualité de code supérieure. Chaque fonctionnalité est validée par un test, et toute régression dans le futur est immédiatement repérée. Les développeurs peuvent refactorer le code sans crainte de casser quelque chose, car les tests veilleront à ce que tout continue à fonctionner correctement.

B. Exercice : Quiz

[solution n°2 p.21]

Question 1

Qu'est-ce que le Test-Driven Development (TDD) ?

- ○ Une méthode qui consiste à écrire du code avant d'écrire des tests
- ○ Une méthode qui consiste à écrire des tests avant d'écrire du code
- ○ Un processus d'intégration continue
- ○ Un type de test manuel

Question 2

Quelle est la première étape du cycle TDD ?

- ○ Écrire le code
- ○ Refactoriser le code
- ○ Écrire un test qui échoue
- ○ Lancer tous les tests

Question 3

Que signifie "Red" dans le cycle Red-Green-Refactor ?

- ○ Une fonctionnalité est complètement implémentée
- ○ Le test échoue parce que la fonctionnalité n'est pas encore codée
- ○ Le code a été testé et validé
- ○ Le code est en cours de refactorisation

Question 4

Tous droits réservés à STUDI - Reproduction interdite

Quelle est la phase "Green" dans le cycle TDD ?

- Écrire du code pour que le test passe
- Supprimer les tests échoués
- Améliorer la structure du code
- Lancer les tests sur le code final

Question 5

Quel est l'objectif principal de la phase "Refactor" dans le cycle TDD ?

- Ajouter de nouvelles fonctionnalités
- Réécrire les tests pour qu'ils réussissent
- Améliorer la qualité du code tout en s'assurant que les tests passent toujours
- Tester la performance du programme

Question 6

Pourquoi le TDD est-il bénéfique pour la qualité du code ?

- Parce qu'il rend les tests optionnels
- Parce qu'il permet d'écrire des tests après avoir codé
- Parce qu'il oblige à écrire des tests exhaustifs avant même de commencer le développement
- Parce qu'il rend la refactorisation plus rapide

Question 7

Dans le TDD, pourquoi écrit-on juste assez de code pendant la phase "Green" ?

- Pour que le test échoue à nouveau
- Pour que le test passe, sans se soucier de la qualité ou des optimisations
- Pour que la fonctionnalité soit totalement terminée
- Pour optimiser immédiatement le code

Question 8

Quelle est la séquence correcte du cycle TDD ?

- Green → Red → Refactor
- Red → Green → Refactor
- Refactor → Red → Green
- Green → Refactor → Red

Question 9

Tous droits réservés à STUDI - Reproduction interdite

Le TDD oblige les développeurs à réfléchir à _____ avant de commencer à coder.

- La refactorisation du code
- La structure des classes
- La manière dont le code sera testé
- L'intégration continue

Question 10

Quel est le résultat final du cycle TDD complet (Red-Green-Refactor) ?

- Un code non testé mais fonctionnel
- Un code testé, fonctionnel et refactorisé
- Un code testé, mais qui ne fonctionne pas toujours
- Un code fonctionnel, mais mal structuré

IV. Introduction à Gherkin et aux tests d'acceptation

A. Introduction à Gherkin et aux tests d'acceptation

Définition **Les tests d'acceptation**

Les tests d'acceptation sont des **méthodes permettant de vérifier qu'une fonctionnalité répond aux exigences métier définies par les utilisateurs ou les parties prenantes**. Contrairement aux tests unitaires qui vérifient des unités de code isolées, les tests d'acceptation valident le comportement global du système.

Le langage Gherkin est souvent utilisé pour rédiger ces tests sous forme de scénarios compréhensibles aussi bien par les développeurs que par les non-techniciens. Gherkin est un **langage spécifique au domaine (DSL)** qui permet de définir des tests d'acceptation en langage naturel.

Définition **Gherkin**

Gherkin est un langage simple qui permet de décrire les comportements attendus du logiciel à l'aide de scénarios. C'est un DSL (Domain-Specific Language), c'est-à-dire un langage spécifique à un domaine, conçu pour **faciliter la communication** entre les développeurs, les testeurs et les parties prenantes non techniques (comme les chefs de projet ou les clients).

Le principal objectif de Gherkin est de décrire le comportement attendu d'une application sous forme de tests lisibles par des humains. En utilisant Gherkin, nous pouvons **rédiger des scénarios qui se concentrent sur le comportement de l'application plutôt que sur les détails techniques.**

Exemple

Imaginons que vous devez écrire un test pour une fonctionnalité de connexion utilisateur. En Gherkin, vous pouvez décrire ce test sous forme de scénario en langage naturel, comme suit :

```
Feature: Connexion utilisateur
  Scenario: L'utilisateur se connecte avec succès
    Given un utilisateur non connecté
    When il entre des identifiants valides
    Then il est redirigé vers son tableau de bord
```


Tous droits réservés à STUDI - Reproduction interdite

Contenu d'un code Gherkin

Les tests en Gherkin suivent une structure bien définie avec des mots-clés spécifiques qui facilitent la rédaction et l'interprétation des tests, dans l'exemple précédent, nous avons utilisé Feature, Scenario, Given, When ainsi que Then :

- **Feature** : ce mot-clé décrit la fonctionnalité à tester. Une Feature est un regroupement de scénarios qui partagent un objectif commun.
- **Scenario** : ce mot-clé représente un cas de test. Chaque scénario décrit une situation spécifique et la manière dont le système devrait se comporter dans cette situation.
- **Given** (étant donné) : il décrit l'état initial du système avant l'action testée. Il s'agit du contexte dans lequel l'action va se dérouler.
- **When** (quand) : ce mot-clé indique l'action ou l'événement qui déclenche le comportement que l'on veut tester. Il représente ce qui est fait par l'utilisateur ou le système.
- **Then** (alors) : il décrit l'attente ou le résultat de l'action. Il valide que le comportement observé est bien celui attendu.

Exécution de tests écrits via Gherkin

Pour exécuter les tests écrits en Gherkin, nous avons besoins d'un outil capable de comprendre ce langage et d'exécuter les scénarios. En Python, l'outil le plus couramment utilisé pour cela est **Behave**. Nous pouvons l'intaller via '*pip install behave*'

Behave est un framework Python qui permet d'exécuter des scénarios décrits en Gherkin. Il associe chaque étape décrite dans Gherkin à une implémentation Python qui exécute les actions correspondantes.

Voici comment Behave fonctionne :

- **Fichier Gherkin** : vous décrivez votre test sous forme de scénario en Gherkin dans un fichier `.feature`.
- **Étapes associées** : pour chaque étape `Given`, `When`, et `Then`, vous écrivez du code Python qui définit ce que l'étape doit faire.
- **Exécution** : behave exécute les scénarios, en suivant les étapes définies.

Voyons ces 3 éléments dans du code python :

Exemple **Test en Gherkin avec implémentation**

Fichier .feature` :

```
Feature: Connexion utilisateur
  Scenario: Connexion réussie avec identifiants valides
    Given un utilisateur non connecté
    When il entre des identifiants valides
    Then il doit être redirigé vers son tableau de bord
```

Implémentation Python (steps.py) :

```python
from behave import given, when, then

@given('un utilisateur non connecté')
def step_impl(context):
    context.utilisateur = Utilisateur()
    context.utilisateur.deconnecter()

@when('il entre des identifiants valides')
def step_impl(context):
    context.utilisateur.connecter("nom_utilisateur", "mot_de_passe")

```

Tous droits réservés à STUDI - Reproduction interdite

```
12 @then('il doit être redirigé vers son tableau de bord')
13 def step_impl(context):
14     assert context.utilisateur.est_connecte()
```

Remarque

Behave associe chaque étape de Gherkin (`Given`, `When`, `Then`) à une fonction Python correspondante. Chaque fonction exécute le code nécessaire pour valider l'étape. Cela permet de tester des fonctionnalités de bout en bout de manière plus intégrée que les simples tests unitaires.

B. Exercice : Quiz

[solution n°3 p.23]

Question 1

Qu'est-ce que Gherkin ?

- ○ Un langage de programmation pour créer des bases de données
- ○ Un langage spécifique au domaine (DSL) utilisé pour écrire des tests en langage naturel
- ○ Un outil pour automatiser les déploiements
- ○ Un framework pour écrire des tests unitaires

Question 2

Que permet de tester Gherkin ?

- ○ Les performances du système
- ○ Le comportement d'une application dans des scénarios spécifiques
- ○ Les tests de charge uniquement
- ○ Les tests de sécurité

Question 3

Quel mot-clé est utilisé dans Gherkin pour décrire l'état initial avant qu'une action ne se produise ?

- ○ When
- ○ Then
- ○ And
- ○ Given

Question 4

Quel mot-clé dans Gherkin est utilisé pour décrire une action effectuée par l'utilisateur ?

- ○ Given
- ○ Then
- ○ When
- ○ And

Question 5

Tous droits réservés à STUDI - Reproduction interdite

Quelle étape décrit le résultat attendu après une action dans un scénario Gherkin ?

- Given
- Then
- When
- And

Question 6

Quelle est la structure correcte d'un fichier Gherkin ?

- Given → Then → When
- When → Then → Given
- Feature → Scenario → Given → When → Then
- Scenario → Feature → Given → Then

Question 7

Que permet le mot-clé Feature dans Gherkin ?

- Décrire un test unitaire
- Décrire une fonctionnalité générale du système à tester
- Décrire l'action d'un utilisateur spécifique
- Décrire l'intégration entre deux modules

Question 8

Quel outil Python permet d'exécuter des scénarios Gherkin ?

- unittest
- Pytest
- Behave
- Selenium

Question 9

Comment Behave gère-t-il les étapes d'un scénario Gherkin ?

- Chaque étape est directement convertie en test unitaire
- Chaque étape est associée à une fonction Python correspondante dans un fichier steps.py
- Behave exécute automatiquement les étapes sans code Python
- Behave remplace les tests unitaires par des tests d'intégration

Question 10

Tous droits réservés à STUDI - Reproduction interdite

Quelle est la principale différence entre les tests unitaires et les tests d'acceptation écrits en Gherkin ?

- Les tests unitaires vérifient le comportement général du système, tandis que les tests Gherkin se concentrent sur une partie spécifique
- Les tests unitaires se concentrent sur des unités isolées de code, tandis que les tests Gherkin valident des scénarios complets basés sur le comportement attendu
- Les tests Gherkin sont plus rapides que les tests unitaires
- Les tests unitaires sont écrits en langage naturel

V. Essentiel

Les tests unitaires permettent de tester des unités isolées de code, comme des fonctions ou des méthodes, pour vérifier qu'elles fonctionnent correctement indépendamment des autres parties du système. Ces tests sont essentiels pour réduire les bugs, faciliter le refactoring et protéger contre les régressions.

Le Test-Driven Development (TDD) va plus loin en structurant le développement autour des tests. En suivant le cycle Red-Green-Refactor, les développeurs écrivent d'abord un test (qui échoue), puis juste assez de code pour que le test passe, avant de refactoriser le code. Le TDD garantit un code testable, modulaire et de meilleure qualité.

Enfin, les tests d'acceptation sont décrits à l'aide du langage Gherkin, qui permet d'écrire des scénarios en langage naturel pour valider le comportement attendu d'une application. Ces scénarios sont exécutés à l'aide d'outils comme Behave en Python, qui relient chaque étape (`Given`, `When`, `Then`) à des implémentations Python. Gherkin facilite la communication entre développeurs et parties prenantes non techniques, en se concentrant sur la validation des exigences fonctionnelles du système.

VI. Auto-évaluation

A. Exercice

José, propriétaire d'un site de commerce en ligne, souhaite améliorer la gestion de son inventaire. Actuellement, il veut s'assurer que lorsqu'un utilisateur achète un produit, celui-ci est bien retiré de l'inventaire. José souhaite aussi ajouter une fonctionnalité permettant de vérifier si un produit est en rupture de stock.

Vous êtes chargé d'implémenter ces deux fonctionnalités en utilisant une approche **TDD** et d'écrire un test d'acceptation en **Gherkin** pour la gestion des produits en stock.

Question 1

[solution n°4 p.26]

José vous demande d'écrire une fonction qui permet de retirer un produit de l'inventaire lorsqu'il est acheté. Vous devez également vérifier que le stock ne devient jamais négatif (le retrait ne peut pas se faire si le stock est insuffisant).

Un produit est défini par un libellé ainsi qu'un nombre définissant sa quantité.

Question 2

[solution n°5 p.27]

José souhaite valider le comportement de son application lorsqu'un produit est en rupture de stock. Vous devez écrire un scénario Gherkin pour cette fonctionnalité, en modélisant la gestion du stock via la classe Produit.

Vous devez donc rédiger un scénario Gherkin pour vérifier que lorsqu'un utilisateur tente d'acheter un produit en rupture de stock, l'achat est refusé et un message est affiché.

B. Test

Exercice 1 : Quiz

[solution n°6 p.28]

Question 1

Tous droits réservés à STUDI - Reproduction interdite

Dans quel cas un test unitaire échoue-t-il ?

- Lorsque le code ne respecte pas les exigences de performance
- Lorsque le résultat réel d'une fonction ne correspond pas au résultat attendu
- Lorsque le code utilise des bibliothèques externes
- Lorsque plusieurs tests sont exécutés simultanément

Question 2

En quoi le TDD améliore-t-il la conception du code ?

- Il empêche les tests d'échouer
- Il oblige à écrire des tests qui conduisent à une conception de code modulaire et testable
- Il force à écrire du code plus rapidement
- Il automatise l'écriture des tests

Question 3

Pourquoi le cycle TDD commence-t-il par un test qui échoue ?

- Pour s'assurer que le test est correctement écrit
- Pour vérifier que la fonctionnalité n'existe pas encore
- Pour éviter d'écrire du code trop rapidement
- Pour passer directement à l'étape Refactor

Question 4

Que fait l'étape assertEqual() dans un test unitaire ?

- Elle compare deux valeurs et vérifie qu'elles sont différentes
- Elle compare deux valeurs et vérifie qu'elles sont égales
- Elle exécute une fonction pour la tester
- Elle vérifie que le test est correctement configuré

Question 5

Dans Behave, que permet de faire l'objet context ?

- Il permet de configurer Behave pour des tests d'intégration
- Il permet de partager des données entre les étapes d'un scénario Gherkin
- Il est utilisé pour définir les tests unitaires dans Behave
- Il sert à créer des rapports de test

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

studi

Exercice p. 5 Solution n°1

Question 1

Quel est l'objectif principal d'un test unitaire ?

- ○ Tester l'intégralité du système
- ◉ Vérifier qu'une unité isolée de code fonctionne correctement
- ○ Vérifier la performance du programme
- ○ Tester l'interaction entre plusieurs modules

Un test unitaire vérifie qu'une petite unité de code, comme une fonction ou une méthode, fonctionne correctement de manière isolée.

Question 2

Quelle bibliothèque est principalement utilisée pour écrire des tests unitaires en Python ?

- ○ NumPy
- ◉ unittest
- ○ Flask
- ○ Pytest

La bibliothèque unittest est intégrée à Python et est couramment utilisée pour écrire des tests unitaires.

Question 3

Pourquoi les tests unitaires sont-ils importants ?

- ○ Ils permettent d'optimiser le code
- ◉ Ils facilitent la détection des bugs à un stade précoce
- ○ Ils augmentent les performances du programme
- ○ Ils réduisent le temps de développement

Les tests unitaires permettent de détecter rapidement des erreurs dans une fonction ou méthode, évitant ainsi la propagation des bugs dans le reste du code.

Question 4

Qu'est-ce que l'instruction assertEqual dans unittest vérifie ?

- ○ Que deux valeurs sont différentes
- ◉ Que deux valeurs sont égales
- ○ Que la fonction soulève une exception
- ○ Que la fonction retourne toujours True

L'instruction assertEqual vérifie que deux valeurs sont égales. Si elles ne le sont pas, le test échoue.

Tous droits réservés à STUDI - Reproduction interdite

Question 5

Quel est le meilleur moment pour exécuter les tests unitaires ?

- ○ Seulement avant de livrer le code en production
- ◉ À chaque modification du code
- ○ Une fois que toutes les fonctionnalités sont implémentées
- ○ Avant de commencer à écrire du code

Q Les tests unitaires doivent être exécutés régulièrement, à chaque modification du code, pour s'assurer que tout fonctionne correctement après les changements.

Question 6

Que se passe-t-il si un test unitaire échoue ?

- ○ Le programme continue à s'exécuter normalement
- ◉ Il lève une exception pour signaler qu'une partie du code ne fonctionne pas comme prévu
- ○ Le test est ignoré
- ○ Le code entier est supprimé

Q Si un test unitaire échoue, il lève une exception, signalant que le résultat du code ne correspond pas au résultat attendu.

Question 7

Quelle est la bonne pratique pour un test unitaire efficace ?

- ○ Tester plusieurs fonctionnalités à la fois
- ◉ Tester chaque fonctionnalité individuellement
- ○ Ignorer les dépendances externes
- ○ Utiliser le même test pour plusieurs fonctions

Q Un test unitaire doit tester une seule fonctionnalité à la fois, de manière isolée, pour identifier précisément ce qui cause un échec.

Question 8

Pourquoi est-il important de maintenir les tests unitaires à jour ?

- ◉ Parce que les tests doivent refléter les changements dans le code
- ○ Parce que les tests obsolètes sont automatiquement supprimés
- ○ Pour augmenter la vitesse des tests
- ○ Parce que Python exige des tests unitarisés pour chaque nouvelle ligne de code

Q Il faut maintenir les tests unitaires à jour, car ils doivent toujours refléter les dernières modifications dans le code pour rester pertinents.

Question 9

Que signifie la notion de « *couverture de code* » dans les tests unitaires ?

Tous droits réservés à STUDI - Reproduction interdite

- ◉ Le pourcentage de lignes de code qui ont été exécutées pendant les tests
- ○ Le nombre total de tests unitaires dans un projet
- ○ La durée totale d'exécution des tests
- ○ Le nombre d'exceptions gérées par le programme

Q La couverture de code représente le pourcentage de lignes de code qui ont été testées lors de l'exécution des tests unitaires, indiquant dans quelle mesure le code est vérifié.

Question 10

Quelle est la bonne façon de structurer un test unitaire ?

- ○ Appeler directement toutes les fonctions dans le test principal
- ◉ Préparer les données, exécuter la fonction à tester, puis valider le résultat
- ○ Tester plusieurs fonctions dans le même test pour gagner du temps
- ○ Ignorer les valeurs d'entrée et se concentrer uniquement sur le résultat

Q Un test unitaire bien structuré suit trois étapes : préparation des données, exécution de la fonction à tester, puis validation des résultats en utilisant des assertions.

Exercice p. 10 Solution n°2

Question 1

Qu'est-ce que le Test-Driven Development (TDD) ?

- ○ Une méthode qui consiste à écrire du code avant d'écrire des tests
- ◉ Une méthode qui consiste à écrire des tests avant d'écrire du code
- ○ Un processus d'intégration continue
- ○ Un type de test manuel

Q Le TDD est une méthode de développement où les tests sont écrits avant le code, et le développement est guidé par ces tests.

Question 2

Quelle est la première étape du cycle TDD ?

- ○ Écrire le code
- ○ Refactoriser le code
- ◉ Écrire un test qui échoue
- ○ Lancer tous les tests

Q La première étape du TDD est d'écrire un test qui échoue (phase Red), car la fonctionnalité n'est pas encore implémentée.

Question 3

Que signifie "Red" dans le cycle Red-Green-Refactor ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ Une fonctionnalité est complètement implémentée
- ◉ Le test échoue parce que la fonctionnalité n'est pas encore codée
- ○ Le code a été testé et validé
- ○ Le code est en cours de refactorisation

Q La phase Red correspond à un test échoué, car la fonctionnalité que l'on veut tester n'a pas encore été codée.

Question 4

Quelle est la phase "Green" dans le cycle TDD ?

- ◉ Écrire du code pour que le test passe
- ○ Supprimer les tests échoués
- ○ Améliorer la structure du code
- ○ Lancer les tests sur le code final

Q La phase Green consiste à écrire juste assez de code pour que le test passe, sans se soucier de la qualité ou de l'optimisation.

Question 5

Quel est l'objectif principal de la phase "Refactor" dans le cycle TDD ?

- ○ Ajouter de nouvelles fonctionnalités
- ○ Réécrire les tests pour qu'ils réussissent
- ◉ Améliorer la qualité du code tout en s'assurant que les tests passent toujours
- ○ Tester la performance du programme

Q La phase Refactor consiste à améliorer la qualité et la structure du code sans changer son comportement, tout en s'assurant que les tests continuent de réussir.

Question 6

Pourquoi le TDD est-il bénéfique pour la qualité du code ?

- ○ Parce qu'il rend les tests optionnels
- ○ Parce qu'il permet d'écrire des tests après avoir codé
- ◉ Parce qu'il oblige à écrire des tests exhaustifs avant même de commencer le développement
- ○ Parce qu'il rend la refactorisation plus rapide

Q Le TDD pousse à écrire des tests exhaustifs avant d'écrire le code, ce qui améliore la couverture des tests et la qualité du code.

Question 7

Dans le TDD, pourquoi écrit-on juste assez de code pendant la phase "Green" ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ Pour que le test échoue à nouveau
- ◉ Pour que le test passe, sans se soucier de la qualité ou des optimisations
- ○ Pour que la fonctionnalité soit totalement terminée
- ○ Pour optimiser immédiatement le code

Q Pendant la phase Green, on écrit juste assez de code pour faire passer le test, sans se soucier de la qualité ou des optimisations à ce stade.

Question 8

Quelle est la séquence correcte du cycle TDD ?

- ○ Green → Red → Refactor
- ◉ Red → Green → Refactor
- ○ Refactor → Red → Green
- ○ Green → Refactor → Red

Q Le cycle TDD suit la séquence Red → Green → Refactor : un test échoue (Red), du code minimal est écrit pour le faire réussir (Green), puis le code est amélioré (Refactor).

Question 9

Le TDD oblige les développeurs à réfléchir à _____ avant de commencer à coder.

- ○ La refactorisation du code
- ○ La structure des classes
- ◉ La manière dont le code sera testé
- ○ L'intégration continue

Q Le TDD pousse les développeurs à réfléchir à la manière dont leur code sera testé avant même d'écrire la première ligne de code. Cela conduit à une conception de code plus modulaire et testable.

Question 10

Quel est le résultat final du cycle TDD complet (Red-Green-Refactor) ?

- ○ Un code non testé mais fonctionnel
- ◉ Un code testé, fonctionnel et refactorisé
- ○ Un code testé, mais qui ne fonctionne pas toujours
- ○ Un code fonctionnel, mais mal structuré

Q Le résultat final du cycle TDD est un code fonctionnel qui a été testé et refactorisé pour améliorer sa qualité et sa structure.

Exercice p. 14 Solution n°3

Tous droits réservés à STUDI - Reproduction interdite

Question 1

Qu'est-ce que Gherkin ?

- ○ Un langage de programmation pour créer des bases de données
- ◉ Un langage spécifique au domaine (DSL) utilisé pour écrire des tests en langage naturel
- ○ Un outil pour automatiser les déploiements
- ○ Un framework pour écrire des tests unitaires

Gherkin est un DSL (Domain-Specific Language) utilisé pour écrire des tests d'acceptation en langage naturel, souvent utilisé dans le cadre de tests BDD.

Question 2

Que permet de tester Gherkin ?

- ○ Les performances du système
- ◉ Le comportement d'une application dans des scénarios spécifiques
- ○ Les tests de charge uniquement
- ○ Les tests de sécurité

Gherkin est utilisé pour **tester le comportement** d'une application via des scénarios décrits en langage naturel.

Question 3

Quel mot-clé est utilisé dans Gherkin pour décrire l'état initial avant qu'une action ne se produise ?

- ○ When
- ○ Then
- ○ And
- ◉ Given

Le mot-clé **Given** est utilisé pour décrire l'état initial du système avant qu'une action ne soit effectuée.

Question 4

Quel mot-clé dans Gherkin est utilisé pour décrire une action effectuée par l'utilisateur ?

- ○ Given
- ○ Then
- ◉ When
- ○ And

Le mot-clé **When** est utilisé pour décrire une action effectuée par l'utilisateur ou un événement déclencheur.

Question 5

Quelle étape décrit le résultat attendu après une action dans un scénario Gherkin ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ Given
- ◉ Then
- ○ When
- ○ And

Le mot-clé **Then** est utilisé pour décrire le **résultat attendu** après qu'une action a été effectuée.

Question 6

Quelle est la structure correcte d'un fichier Gherkin ?

- ○ Given → Then → When
- ○ When → Then → Given
- ◉ Feature → Scenario → Given → When → Then
- ○ Scenario → Feature → Given → Then

La structure correcte est **Feature → Scenario → Given → When → Then**.

Question 7

Que permet le mot-clé Feature dans Gherkin ?

- ○ Décrire un test unitaire
- ◉ Décrire une fonctionnalité générale du système à tester
- ○ Décrire l'action d'un utilisateur spécifique
- ○ Décrire l'intégration entre deux modules

Feature est utilisé pour décrire une fonctionnalité générale à tester dans le système.

Question 8

Quel outil Python permet d'exécuter des scénarios Gherkin ?

- ○ unittest
- ○ Pytest
- ◉ Behave
- ○ Selenium

Behave est un framework Python utilisé pour exécuter des tests d'acceptation écrits en Gherkin.

Question 9

Comment Behave gère-t-il les étapes d'un scénario Gherkin ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ Chaque étape est directement convertie en test unitaire
- ◉ Chaque étape est associée à une fonction Python correspondante dans un fichier steps.py
- ○ Behave exécute automatiquement les étapes sans code Python
- ○ Behave remplace les tests unitaires par des tests d'intégration

Q Chaque étape Gherkin (Given, When, Then) est associée à une fonction Python correspondante dans un fichier steps.py, qui implémente l'action à réaliser.

Question 10

Quelle est la principale différence entre les tests unitaires et les tests d'acceptation écrits en Gherkin ?

- ○ Les tests unitaires vérifient le comportement général du système, tandis que les tests Gherkin se concentrent sur une partie spécifique
- ◉ Les tests unitaires se concentrent sur des unités isolées de code, tandis que les tests Gherkin valident des scénarios complets basés sur le comportement attendu
- ○ Les tests Gherkin sont plus rapides que les tests unitaires
- ○ Les tests unitaires sont écrits en langage naturel

Q Les tests unitaires se concentrent sur des unités spécifiques de code, tandis que les tests Gherkin valident le **comportement global** à travers des scénarios complets.

p. 16 **Solution n°4**

Étape 1 : Écriture des tests unitaires (Phase Red)

```python
import unittest

class TestProduit(unittest.TestCase):

    def test_retirer_produit(self):
        produit = Produit("ProduitA", 10)
        produit.retirer_produit(4)
        self.assertEqual(produit.stock, 6)

    def test_retrait_insuffisant(self):
        produit = Produit("ProduitA", 5)
        with self.assertRaises(ValueError):
            produit.retirer_produit(6)

if __name__ == "__main__":
    unittest.main()
```

Étape 2 : Implémentation de la classe Produit (Phase Green)

```python
class Produit:
    def __init__(self, nom, stock):
        self.nom = nom
        self.stock = stock

    def retirer_produit(self, quantite):
        if quantite > self.stock:
            raise ValueError("Stock insuffisant pour retirer cette quantité")
        self.stock -= quantite

```


Tous droits réservés à STUDI - Reproduction interdite

Explication :

- Le test **test_retirer_produit** vérifie que la méthode retirer_produit soustrait correctement la quantité demandée du stock.
- Le test **test_retrait_insuffisant** vérifie que si la quantité demandée est supérieure au stock disponible, la méthode lève une exception.

p. 16 Solution n°5

```
Feature: Gestion du stock
  Scenario: Tentative d'achat d'un produit en rupture de stock
    Given un produit "ProduitB" avec 0 en stock
    When l'utilisateur essaie d'acheter "ProduitB"
    Then l'achat est refusé
    And un message "Produit en rupture de stock" est affiché
```

Explication :

- **Given** : le produit est en rupture de stock (quantité à 0).
- **When** : l'utilisateur essaie de l'acheter malgré la rupture de stock.
- **Then** : l'achat est refusé.
- **And** : un message indiquant que le produit est en rupture de stock est affiché.

Nous devons ensuite effectuer une implémentation en Python

```python
from behave import given, when, then
from produit import Produit  # La classe Produit est importée depuis votre fichier produit.py

@given('un produit "{nom_produit}" avec {stock:d} en stock')
def step_given_produit_en_stock(context, nom_produit, stock):
    context.produit = Produit(nom_produit, stock)  # Création du produit avec le stock donné

@when('l\'utilisateur essaie d\'acheter "{nom_produit}"')
def step_when_utilisateur_achete_produit(context, nom_produit):
    try:
        context.produit.retirer_produit(1)  # On tente d'acheter 1 unité du produit
        context.achat_reussi = True
    except ValueError as e:
        context.achat_reussi = False
        context.message = str(e)  # On capture le message d'erreur

@then('l\'achat est refusé')
def step_then_achat_refuse(context):
    assert not context.achat_reussi, "L'achat aurait dû être refusé"

@then('un message "{message}" est affiché')
def step_then_message_affiche(context, message):
    assert context.message == message, f"Message attendu: {message}, mais obtenu: {context.message}"
```

Explication des étapes :

1. **@given('un produit "{nom_produit}" avec {stock:d} en stock')**

   Cette étape crée un produit avec le stock initial défini dans le scénario Gherkin. Le produit est stocké dans le context afin qu'il soit accessible dans les étapes suivantes.

Tous droits réservés à STUDI - Reproduction interdite

2. **@when('l\'utilisateur essaie d\'acheter "{nom_produit}"')**

   Cette étape simule la tentative d'achat d'un produit. Si le stock est insuffisant, la méthode retirer_produit() lève une exception (ValueError), et nous stockons l'erreur dans context.message.

3. **@then('l\'achat est refusé')**

   Cette étape vérifie que l'achat a bien été refusé, ce qui signifie que le test s'attend à ce que context.achat_reussi soit False.

4. **@then('un message "{message}" est affiché')**

   Cette étape compare le message d'erreur attendu (dans ce cas "Produit en rupture de stock") avec le message capturé dans context.message.

Exercice p. 16 Solution n°6

Question 1

Dans quel cas un test unitaire échoue-t-il ?

- ○ Lorsque le code ne respecte pas les exigences de performance
- ◉ Lorsque le résultat réel d'une fonction ne correspond pas au résultat attendu
- ○ Lorsque le code utilise des bibliothèques externes
- ○ Lorsque plusieurs tests sont exécutés simultanément

Un test unitaire échoue lorsque le résultat réel d'une fonction ne correspond pas au résultat attendu défini dans le test.

Question 2

En quoi le TDD améliore-t-il la conception du code ?

- ○ Il empêche les tests d'échouer
- ◉ Il oblige à écrire des tests qui conduisent à une conception de code modulaire et testable
- ○ Il force à écrire du code plus rapidement
- ○ Il automatise l'écriture des tests

Le TDD pousse à écrire des tests avant le code, ce qui conduit à une conception de code plus modulaire et plus facile à tester.

Question 3

Pourquoi le cycle TDD commence-t-il par un test qui échoue ?

- ○ Pour s'assurer que le test est correctement écrit
- ◉ Pour vérifier que la fonctionnalité n'existe pas encore
- ○ Pour éviter d'écrire du code trop rapidement
- ○ Pour passer directement à l'étape Refactor

Le cycle TDD commence par un test qui échoue (phase Red) pour vérifier que la fonctionnalité n'est pas encore implémentée, et que le test est prêt pour être validé par l'ajout du code.

Tous droits réservés à STUDI - Reproduction interdite

Question 4

Que fait l'étape assertEqual() dans un test unitaire ?

- ○ Elle compare deux valeurs et vérifie qu'elles sont différentes
- ◉ Elle compare deux valeurs et vérifie qu'elles sont égales
- ○ Elle exécute une fonction pour la tester
- ○ Elle vérifie que le test est correctement configuré

L'étape assertEqual() compare deux valeurs dans un test unitaire et vérifie qu'elles sont égales.

Question 5

Dans Behave, que permet de faire l'objet context ?

- ○ Il permet de configurer Behave pour des tests d'intégration
- ◉ Il permet de partager des données entre les étapes d'un scénario Gherkin
- ○ Il est utilisé pour définir les tests unitaires dans Behave
- ○ Il sert à créer des rapports de test

L'objet context dans Behave est utilisé pour **partager des données** entre les différentes étapes d'un scénario, ce qui permet de maintenir l'état entre Given, When et Then.

Tous droits réservés à STUDI - Reproduction interdite